

《企业级软件实践与工程能力提升训练营》

项目报告

系统名称	店小智——电商智能客服系统
学 院	软件学院
专业班级	软件 2306 班
姓 名	王一帆
学 号	2023005141
指导教师	董凯凯
答辩日期	2026 年 10 月 日

目 录

（请在本页通过 Word“引用 → 目录 → 自动目录”插入目录，或右键下方目录域选择“更新域”）

（目录内容：更新域后自动生成）

1 需求分析

1.1 项目背景与意义

随着电商行业的发展，客服咨询量持续增长，传统人工客服存在响应慢、夜间无人值守、重复问题占用大量人力等问题；直接用通用大模型网页版回答用户，又存在数据不在自己库中、无法查询订单等真实业务数据、容易“一本正经地编造”（幻觉）等缺陷。因此本系统以“企业自己的智能客服”为目标，基于大模型 API 与 LangChain 框架构建了电商智能客服系统“店小智”：它能够记住每位用户的历史对话并持久化保存，能够自主调用订单、天气、计算、时间、优惠券等工具获取真实数据，并通过意图路由、流式输出、限流等工程手段保证体验与安全，最终以 Docker 容器化方式一键交付。

1.2 用户与业务场景

场景一：售前咨询。潜在顾客深夜浏览商品，想了解发货时效，直接询问“长沙今天天气怎么样？适合发货吗？”，系统自动调用天气工具给出真实天气并结合发货规则回答，无需人工值班。

场景二：订单查询。老顾客回访时间“订单 DD20240001 现在到哪了？”，智能体调用订单查询工具返回物流状态；顾客接着说“顺便帮我算下 365*24*60，一年有多少分钟”，智能体在同一会话中记忆上下文并接力调用计算工具。

场景三：日常问答与优惠咨询。用户闲聊“用一句话介绍 Docker”，系统走普通对话链路快速响应；用户再问“优惠券 QUAN100 还能用吗”，意图路由自动把该请求分流到智能体链路查询优惠券真实状态。

1.3 功能需求

表 1 功能需求清单

功能模块	功能描述	优先级
多轮对话	系统记住用户上下文，支持连续问答，支持 SSE 流式逐字输出	核心
会话管理	会话列表/新建/删除/改名/历史消息恢复，首条消息	核心

功能模块	功能描述	优先级
	自动生成标题	
智能体工具调用	订单、天气、计算、时间、优惠券 5 个工具的自主调用与多步编排	核心
智能体记忆持久化	SqliteSaver 检查点保存智能体多轮状态，服务重启不丢	核心
意图路由统一入口	结构化输出识别用户意图，自动分流“闲聊/需工具”两类链路	重要
接口防护	每 IP 每分钟 20 次滑动窗口限流，超限返回 429	重要
容器化交付	docker compose 一键部署，数据卷持久化，支持毁灭测试	核心
评估体系	自动化评估脚本度量工具选择正确率与平均响应延迟	加分

1.4 非功能需求

性能：普通对话接口平均响应时间 ≤ 3 秒；智能体接口（需多次模型往返）平均响应时间 ≤ 8 秒；SSE 流式接口首字返回时间 ≤ 3 秒。

安全：大模型 API Key 通过 .env 环境变量管理，严禁写入代码与 Git 仓库；计算工具采用字符白名单过滤防止代码注入；每 IP 每分钟最多 20 次 /api 请求，超限返回 429；接口异常统一兜底，不向前端暴露堆栈信息。

可用性：所有接口返回统一格式 {code, message, data}；服务重启、容器销毁重建后，会话与聊天记录不丢失。

可维护性：后端按路由层/业务层/数据层分层，配置集中在 config.py；接口文档由 Swagger 自动生成；提供自动化评估脚本，可重复度量智能体质量。

2 系统设计

2.1 总体架构设计

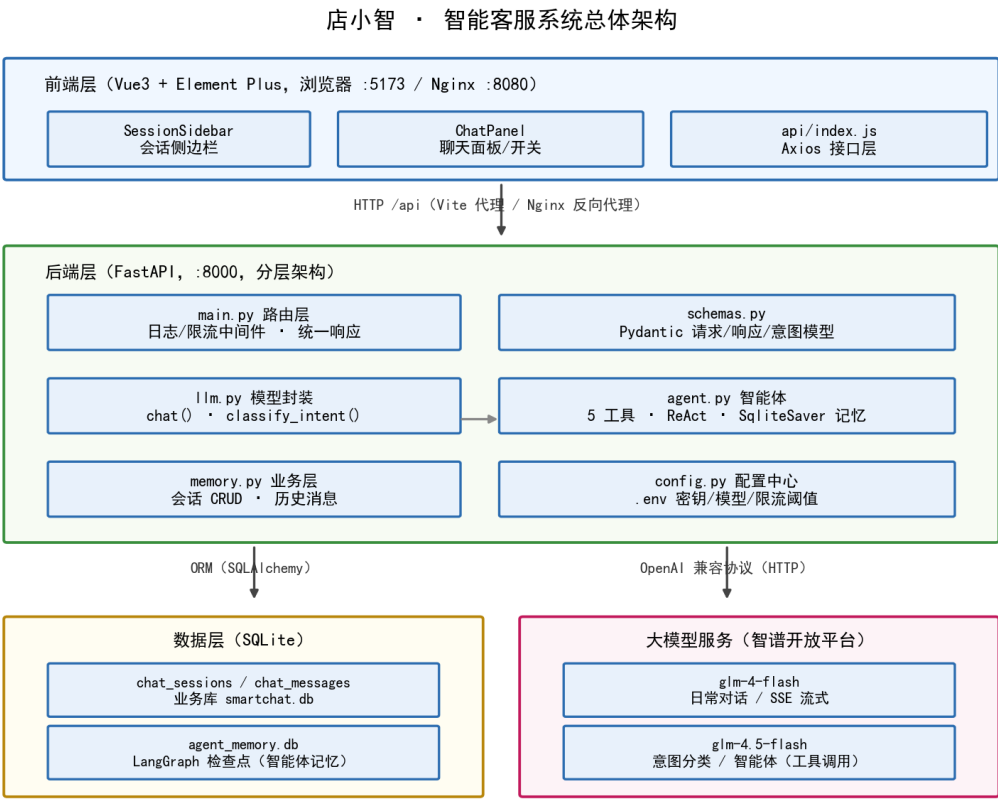


图 1 系统总体架构图

系统总体架构如图 1 所示，自上而下分为四层。前端层基于 Vue3 与 Element Plus，负责会话侧边栏与聊天面板的展示，所有请求经 Vite 代理（开发）或 Nginx 反向代理（部署）进入后端；后端层基于 FastAPI，按“路由层—业务层—数据层”分层 main.py 承载路由与日志、限流中间件，llm.py 封装大模型调用与意图分类，agent.py 实现智能体，memory.py 负责会话与消息存取，config.py 集中管理全部配置；数据层使用 SQLite 保存业务数据（会话/消息）与智能体检查点两套数据；大模型服务采用智谱开放平台的 OpenAI 兼容接口，更换供应商只需修改 .env 中的 BASE_URL 与模型名，代码零改动。

2.2 技术选型与理由

表 2 技术选型表

技术/框架	用途	选型理由
FastAPI	后端 Web 框架	自动生成 Swagger 文档、Pydantic 参数强校验、原生支持异步与 SSE，相比 Flask 免去手写校验与文档
SQLAlchemy + SQLite	ORM 与业务数据库	单文件零部署；ORM 屏蔽 SQL 细节，数据量增大后仅需修改连接串即可迁移 MySQL/PostgreSQL
LangChain 1.x	大模型应用框架	消息对象、@tool 工具注册、with_structured_output 结构化输出标准化，避免手写 Function Calling 轮子
LangGraph + SqliteSaver	智能体运行时与检查点	create_agent 开箱即用 ReAct 循环；检查点把多轮状态落库，重启不丢记忆
智谱 glm-4-flash / glm-4.5-flash	大模型服务	OpenAI 兼容协议避免供应商锁定；两模型均有免费额度；4.5 系工具调用与多轮召回更稳
Vue3 + Element Plus	前端框架与组件库	数据驱动视图，聊天界面所需按钮/输入框/开关等组件齐全，开发效率高
Vite + Axios	前端构建与请求	热更新提升开发体验；开发期 /api 代理解决跨域，与部署期 Nginx 反代同构
Docker + Nginx	容器化交付	环境一致性；前端多阶段构建后由 Nginx 托管并反向代理 /api（关闭缓冲以支持 SSE）

2.3 数据结构设计

表 3 核心数据表设计

字段名	类型	约束	说明
chat_sessions.id	TEXT	主键 (uuid)	会话唯一标识
chat_sessions.title	TEXT	非空，默认“新对话”	会话标题，首条消息后自动截取生成
chat_sessions.created_at / updated_at	DATETIME	自动维护	创建时间 / 最近活跃时间（列表排序依据）
chat_messages.id	INTEGER	主键，自增	消息唯一标识
chat_messages.session_id	TEXT	外键 → chat_sessions.id，级联删除，索引	所属会话
chat_messages.role	TEXT	非空	取值 user / assistant
chat_messages.content	TEXT	非空	消息内容（含工具调用记录前缀）

字段名	类型	约束	说明
ntent			
chat_messages.created_at	DATETIME	默认当前时间	消息时间

会话与消息为一对多关系：一个 chat_sessions 记录对应多条 chat_messages 记录，删除会话时通过外键 ON DELETE CASCADE 级联删除其全部消息。此外系统还有第二个数据库文件 agent_memory.db，由 LangGraph 的 SqliteSaver 托管，以检查点（checkpoint）形式保存智能体每一轮的完整消息状态，供模型跨轮次读取；业务库与检查点库各司其职、互不干扰。

2.4 接口设计

表 4 主要接口清单

接口路径	方法	功能说明
GET /	GET	健康检查，返回服务运行状态
/api/sessions	GET / POST	会话列表 / 新建会话
/api/sessions/{id}	DELETE / PATCH	删除会话（级联删消息）/ 修改会话标题
/api/sessions/{id}/messages	GET	获取某会话全部历史消息（切换会话恢复记录）
/api/chat	POST	普通多轮对话：带历史调模型，结果落库
/api/chat/stream	POST	SSE 流式对话：逐字推送，完成后落库
/api/agent/chat	POST	智能体对话：工具调用 + 记忆持久化，返回 tools_used
/api/smart/chat	POST	统一入口：意图识别后自动路由到普通对话或智能体

全部接口均返回统一响应格式 {"code": 0, "message": "success", "data": {...}}，并在 Swagger (/docs) 中自动生成可交互测试的接口文档，如图 6 所示。

3 核心功能实现

3.1 功能模块划分

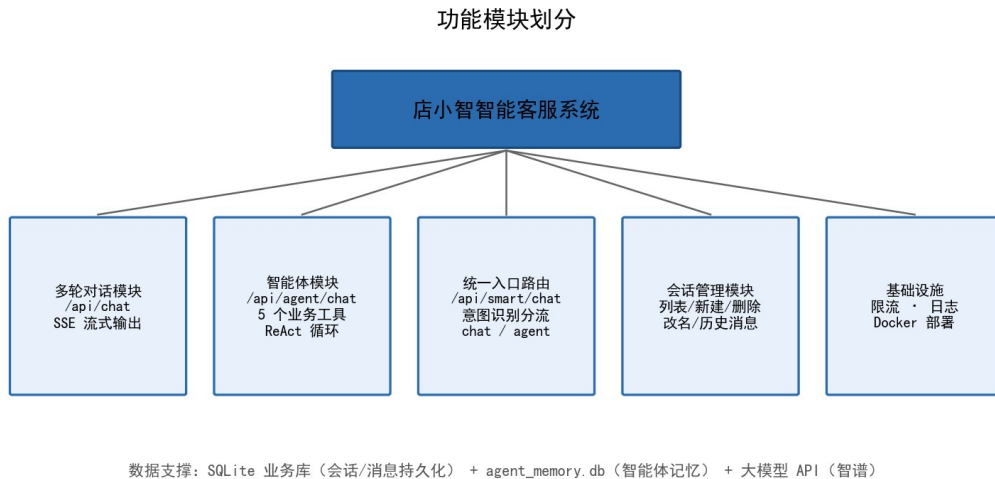


图2 功能模块图

系统功能模块划分如图2所示，与表1功能清单一一对应：前端两个组件与会话状态中枢 App.vue 组成交互层；后端由多轮对话、智能体、意图路由、会话管理四个业务模块与限流、日志、部署三项基础设施组成，全部模块共享 SQLite 业务库、LangGraph 检查点库与智谱大模型服务。

3.2 关键功能实现

3.2.1 功能1：多轮对话与会话记忆持久化

实现思路：

大模型本身没有记忆，每次请求都是“第一次见你”。多轮对话的本质是：程序把该会话的全部历史消息与本轮问题一起发给模型。实现上分三步：①前端每次对话携带 session_id；②后端按 session_id 从 SQLite chat_messages 表取出历史消息，转成 LangChain 消息对象，按“系统提示词→历史→本轮问题”的顺序拼装；③得到回复后把本轮问答写回数据库，并维护会话标题与活跃时间。数据流：用户消息 →

ChatRequest 校验 → memory.get_history() → llm.chat() → memory.append() 落库 → 统一响应返回。

关键代码：

backend/app/llm.py（多轮对话核心：历史一起发给模型）：

```
def chat(user_message, history=None) -> str:
    messages = [SystemMessage(content=SYSTEM_PROMPT)] # 1. 人设
    if history:
        messages.extend(to_messages(history)) # 2. 历史：查库所得
    messages.append(HumanMessage(content=user_message)) # 3. 本轮问题
    return llm.invoke(messages).content # 历史一起发给模型 => 多轮对话

# backend/app/memory.py: 一轮问答写入数据库（持久化）
def append(db, session_id, user_msg, assistant_msg) -> None:
    session = db.get(ChatSession, session_id)
    if not session:
        return
    db.add(ChatMessage(session_id=session_id, role="user", content=user_msg))
    db.add(ChatMessage(session_id=session_id, role="assistant",
        content=assistant_msg))
    if session.title == "新对话" and user_msg: # 首条消息自动生成标题
        session.title = user_msg[:16] + ("..." if len(user_msg) > 16 else "")
    db.commit()
```

运行效果：

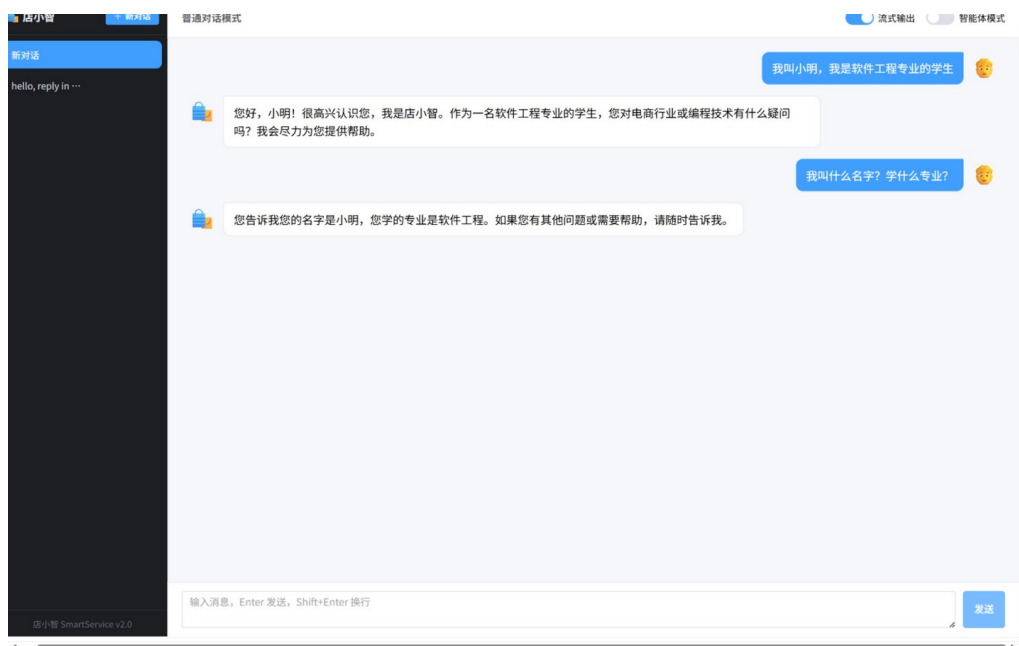


图3 多轮对话记忆效果（AI 正确答出“小明、软件工程”，左侧会话标题自动生成）

3.2.2 功能 2：智能体工具调用与多步任务编排

实现思路：

大模型本质是“文字接龙机器”，对实时数据与精确计算不可靠。智能体方案：大模型当“大脑”负责决策，工具当“手脚”负责执行。用 `@tool` 装饰器把普通函数注册为工具，函数 `docstring` 即“给模型看的说明书”，写清功能、参数格式与触发条件；再用 `create_agent` 组装“模型+工具+系统提示词”，ReAct（思考→行动→观察→再思考）循环由框架内部自动完成：模型返回工具调用意图 JSON，程序真正执行函数并把观察结果回传，模型基于真实结果作答；复合问题会自动拆解为多步、依次调用多个工具。智能体每轮状态由 `SqliteSaver` 检查点落库，实现跨轮次记忆与重启恢复。

关键代码：

backend/app/agent.py（工具注册与智能体组装）：

```
@tool
def query_order(order_id: str) -> str:
    """根据订单号查询物流状态。order_id 格式如 "DD20240001"。
    用户询问订单进度、物流、发货、签收时必须使用本工具。""" # 说明书
    orders = {"DD20240001": "已发货，顺丰速运，预计明天 18:00 前送达", ...}
    return orders.get(order_id, f"未找到订单 {order_id}，请核对订单号")

agent = create_agent(
    model=agent_llm,                # 温度 0：工具调用要稳定
    tools=TOOLS,                    # 5 个工具：订单/天气/计算/时间/优惠券
    system_prompt="你是「店小智」……必须调用工具，禁止编造",
    checkpoint=SqliteSaver(_conn), # 记忆持久化：thread_id 共享记忆
)
result = agent.invoke(
    {"messages": [{"role": "user", "content": message}]},
    config={"configurable": {"thread_id": session_id}})
```

运行效果：



图 4 智能体工具调用效果（回复前缀展示本轮调用的工具名）

图 5 多步任务编排效果（同一问题依次调用 `get_current_time` 与 `calculate`）

3.2.3 功能 3：意图路由统一入口

实现思路：

若让用户自己判断“要不要开智能体模式”，体验差；若全部请求都走智能体，闲聊也要经历多次模型往返，慢且费 Token。因此设计统一入口 `/api/smart/chat`：先用 `with_structured_output` 绑定 Intent 模型对用户消息做意图分类（`chat`=闲聊，`agent`=需要工具），分类结果为强类型 Pydantic 对象；再按结果自动分发到普通对话或智能体链

路。分类模型温度设为 0 保证稳定，并配置关键词规则兜底：分类模型异常或返回异常值时，命中订单/天气/计算等关键词即走智能体链路，保证不漏真实业务请求。

backend/app/llm.py（结构化输出意图分类 + 规则兜底）：

```
class Intent(BaseModel): # schemas.py: 字段即格式
    intent: Literal["chat", "agent"] = Field(
        description="chat=闲聊; agent=需工具的请求")

    @field_validator("intent", mode="before")
    def _normalize(cls, v): # 容错: 兼容 "Agent"/"agent." 等
        s = str(v).strip().lower()
        return "agent" if "agent" in s else "chat"

# llm.py: 显式走函数调用通道, 兼容国产模型
_intent_llm = ChatOpenAI(..., temperature=0).with_structured_output(
    Intent, method="function_calling")

def classify_intent(message: str) -> str:
    try:
        result = _intent_llm.invoke([...])
        if result is not None:
            return result.intent # 正常时直接采用
    except Exception as e:
        logger.warning(f"分类失败, 启用规则兜底: {e}")
    if any(k in message for k in _AGENT_KEYWORDS): # 关键词兜底
        return "agent"
    return "chat"
```

运行效果：

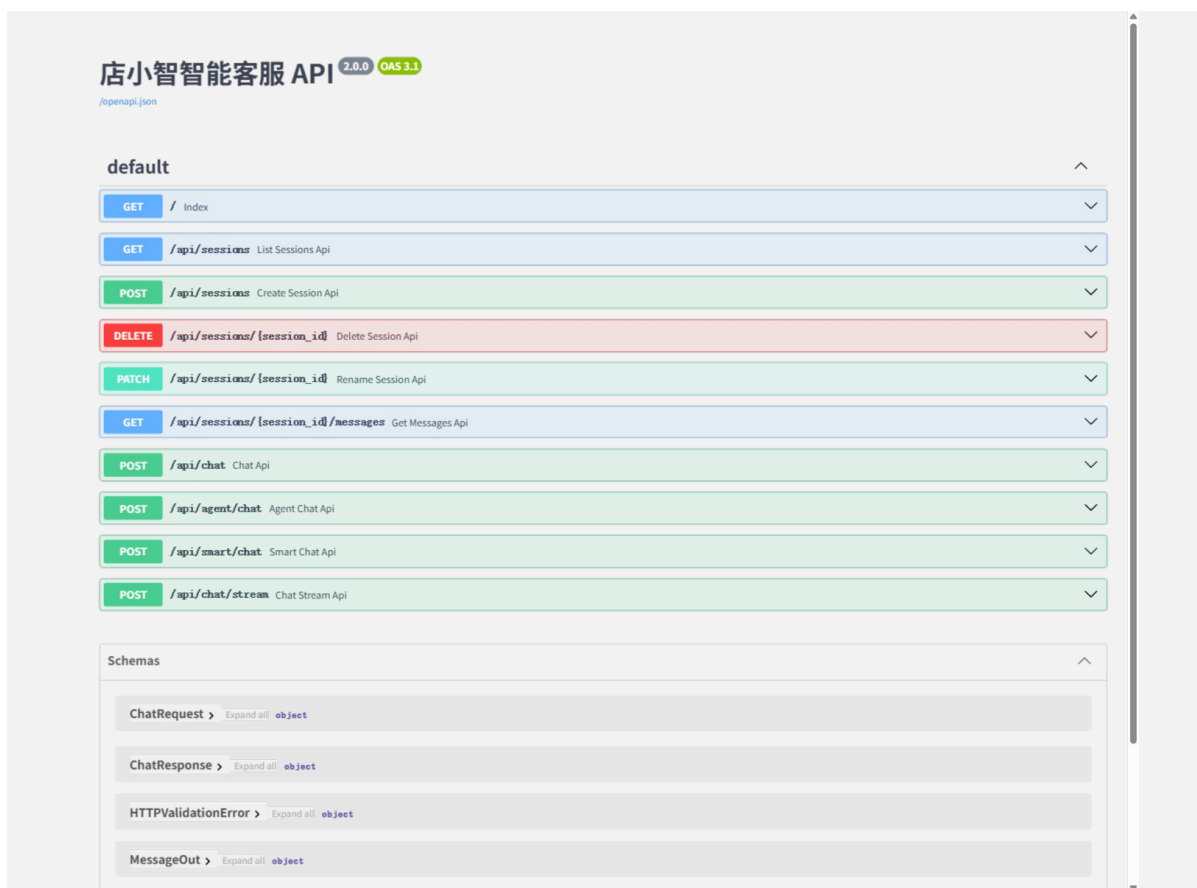


图6 Swagger 接口文档（统一入口 `/api/smart/chat` 可在页面直接测试）

3.2.4 功能4：SSE 流式输出

实现思路：

普通接口要等大模型把整段回复生成完才一次性返回，用户盯着空白等待。SSE（Server-Sent Events）是服务器向浏览器单向持续推送的标准协议：后端用 `llm.astream()` 逐块接收生成内容，每收到一块立即以 `data: {...}` 事件推给前端，全部完成后把完整回复落库并发送 `[DONE]` 结束信号；前端用 `fetch + ReadableStream` 逐块读取并拼接（浏览器原生 `EventSource` 只支持 GET，POST 流式必须手动读流），实现打字机效果。Nginx 反代需关闭 `proxy_buffering` 才能透传 SSE。

backend/app/main.py（后端 SSE 接口）：

```
@app.post("/api/chat/stream")
async def chat_stream_api(req: ChatRequest, db: Session = Depends(get_db)):
    messages = [SystemMessage(content=SYSTEM_PROMPT)]
    messages.extend(to_messages(memory.get_history(db, req.session_id)))
    messages.append(HumanMessage(content=req.message))
```

```
async def event_generator():
    full_reply = ""
    async for chunk in llm.astream(messages): # 异步流式逐块接收
        if chunk.content:
            full_reply += chunk.content
            # SSE 格式: data: {...}\n\n, 逐块推送给前端
            yield f"data: {json.dumps({'delta': chunk.content},
ensure_ascii=False)}\n\n"
    memory.append(db, req.session_id, req.message, full_reply) # 落库
    yield "data: [DONE]\n\n" # 结束信号

return StreamingResponse(event_generator(),
                        media_type="text/event-stream")
```

运行效果:

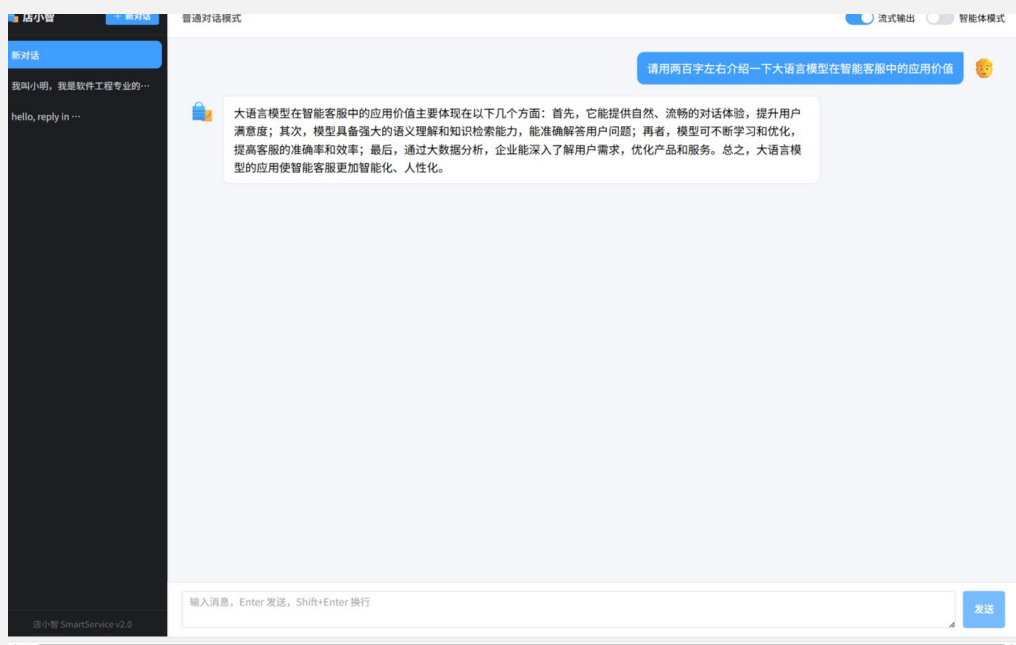


图7 SSE 流式输出效果（回复生成中途截图，可见内容正在逐字追加）

4 测试与优化

4.1 测试方案与用例

表 5 测试用例表

编号	测试项	输入/操作	预期结果	实际结果	结论
T01	健康检查	GET /	返回运行中提示	返回运行中提示	通过
T02	多轮记忆	先说“我叫小明”，再问“我叫什么”	答出小明	答出小明	通过
T03	会话持久化	重启后端后查询历史消息	历史消息仍在	历史消息仍在	通过
T04	工具调用-订单	智能体问“订单 DD20240001 到哪了”	调用 query_order 并答出物流	调用 query_order，答出顺丰已发货	通过
T05	工具调用-优惠券	问“优惠券 QUAN100 还能用吗”	调用 get_coupon	调用 get_coupon，答出满 100 减 20 可用	通过
T06	多步编排	问“现在几点，顺便算 365*24*60”	依次调用时间与计算工具	get_current_time、calculate 接力完成	通过
T07	意图路由	闲聊句与业务句各发一次统一入口	route 分别为 chat / agent	4 组样本全部分流正确	通过
T08	流式输出	开启流式开关提问	逐字追加，收到 [DONE]	12 个数据块逐字推送	通过
T09	参数校验	POST /api/chat, message 为空串	422 校验错误	422，非法数据未进入业务代码	通过
T10	接口限流	1 分钟内连续请求 /api 超过 20 次	超限返回 429	窗口计满后返回 429	通过
T11	智能体评估	运行 test_eval.py 12 个用例	工具选择正确率 $\geq 90\%$	12/12=100%，平均延迟 3174ms	通过
T12	毁灭测试	docker compose down 后重新 up	历史会话仍在	数据卷挂载，历史会话完整保留	通过

4.2 测试结果分析

功能测试共设计 12 个用例（表 5），全部通过，通过率 100%。其中：多轮记忆、持久化、参数校验、限流等确定性用例全部一次通过；智能体相关用例经“评估→修改→重测”迭代后全部通过。

量化指标（来自自动化评估脚本 `test_eval.py`，12 个用例真实调用大模型）：工具选择正确率 $12/12 = 100\%$ （目标 $\geq 90\%$ ）；平均响应延迟 `3174ms`，其中闲聊类不调工具用例延迟最低（约 `700ms`），复杂问句延迟较高（最大 `18s`，为模型长思考所致），符合“智能体接口耗时约为普通对话数倍”的预期。

意图路由抽查：“用一句话介绍 Docker”→chat、“帮我算 $123*456$ ”→agent、“写一首关于春天的诗”→chat、“订单 DD20240002 什么状态”→agent，4 组样本分流全部正确。

4.3 问题与优化

问题一：结构化输出在国产模型上返回 `None` 或校验失败。

问题现象 / 原因分析 / 解决方案：调用 `/api/smart/chat` 时先后出现“Invalid JSON”与“'NoneType' object has no attribute 'intent'”两类异常。排查发现两个原因：一是 `with_structured_output` 默认的 JSON 模式下 glm 系列模型不遵守 `response_format` 约束，直接输出自然语言；二是部分模型未强制执行 `tool_choice`，函数调用通道下不返回工具调用。

解决方案：显式指定 `method="function_calling"` 走函数调用通道；为 Intent 模型增加 `field_validator` 归一化校验器（兼容大小写与标点差异）；再增加关键词规则兜底。修复后意图路由样本测试 4/4 正确。

问题二：glm-4-flash 智能体多轮记忆召回失败。

问题现象 / 原因分析 / 解决方案：同一会话内查完订单后追问“刚才聊的第一个订单号是多少”，模型回答“无法回答”。通过最小复现脚本对比发现：检查点记忆本身写入正常（工具状态完整），是 glm-4-flash 对多轮上下文的召回能力不足。

解决方案：把智能体与意图分类器切换为同为免费档的 glm-4.5-flash（配置中心新增 `AGENT_MODEL/ROUTER_MODEL` 配置项），普通对话仍用更快的 glm-4-flash。复测记忆召回正确答出 DD20240001，12 个评估用例全部通过。

问题三：智能体回复污染与工具记录累积。

问题现象 / 原因分析 / 解决方案：glm-4.5-flash 是思考模型，回复中混入 `</think>` 思考痕迹；同时因 SqliteSaver 的 `invoke` 返回整个线程的全量消息，工具调用记录会把历史轮次也统计进来（`tools_used` 越来越长）。

解决方案：调用智谱 API 时通过 `extra_body` 关闭思考模式，并在 `agent_chat` 中对回复做清洗兜底；工具统计只取最后一条用户消息之后的消息（即仅本轮），并对重复调用去重。修复后回复干净、工具记录准确。

5 部署方案

5.1 部署架构

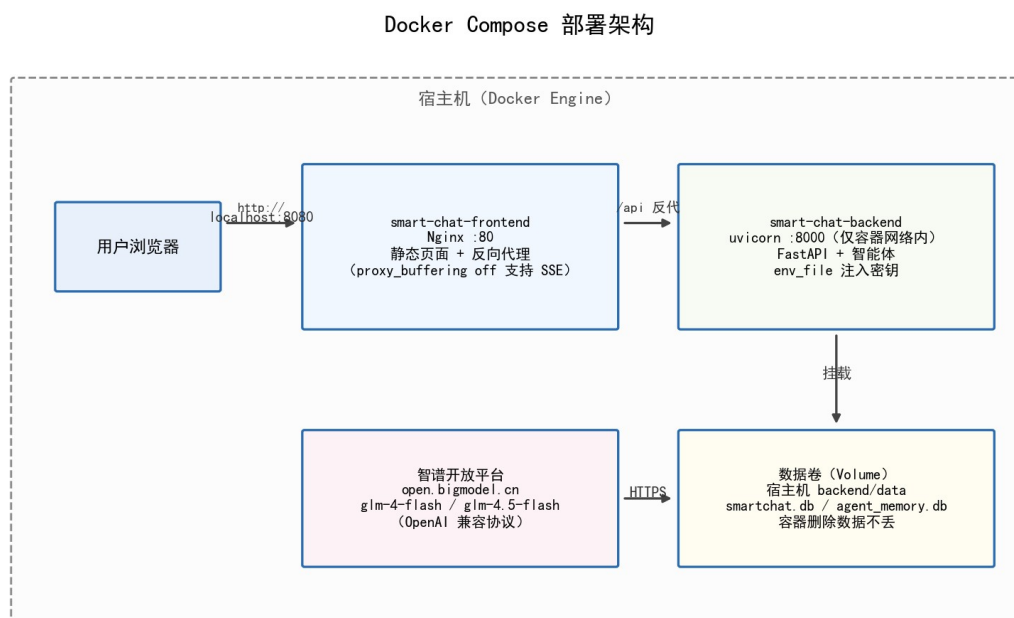


图 8 Docker Compose 部署架构图

系统采用 Docker Compose 双容器部署，部署架构如图 8 所示：frontend 容器（Nginx:80，映射宿主机 8080）托管 Vue 打包后的静态文件，并把 /api 请求反向代理给 backend 容器（uvicorn:8000，仅在容器网络内开放）；API Key 通过 env_file 注入环境变量，不写入镜像；backend/data 目录以数据卷挂载到宿主机，容器销毁重建后 SQLite 数据不丢。Nginx 关闭 proxy_buffering 以支持 SSE 流式响应。

5.2 部署步骤

步骤一（环境准备）：

安装 Docker Desktop（Windows 需 WSL2）；获取代码后执行 cp backend/.env.example backend/.env，在 .env 中填入智谱 API Key（LLM_API_KEY）并按需修改模型名；确认 .gitignore 已排除 backend/.env 与 backend/data/。

步骤二（构建与启动）：

在项目根目录执行 `docker compose up -d --build`：首次构建自动完成前端 `npm build`、后端 `pip install` 并启动两个容器；可用 `docker compose ps` 查看状态、`docker compose logs -f backend` 查看实时日志。

步骤三（验证）：

浏览器访问 `http://localhost:8080` 出现聊天页面即部署成功；在页面中新建对话并提问，验证对话、智能体、流式输出等功能；接口层面可访问 `http://localhost:8080/docs` 查看 Swagger 文档。

5.3 部署验证

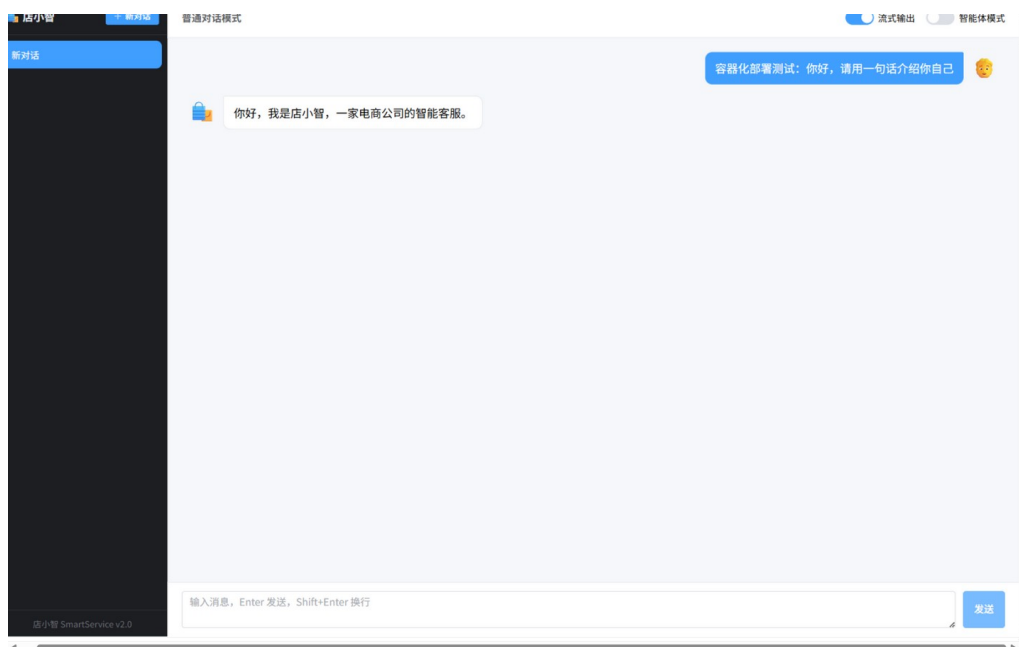


图9 容器化部署后访问 `http://localhost:8080`（页面与功能正常）

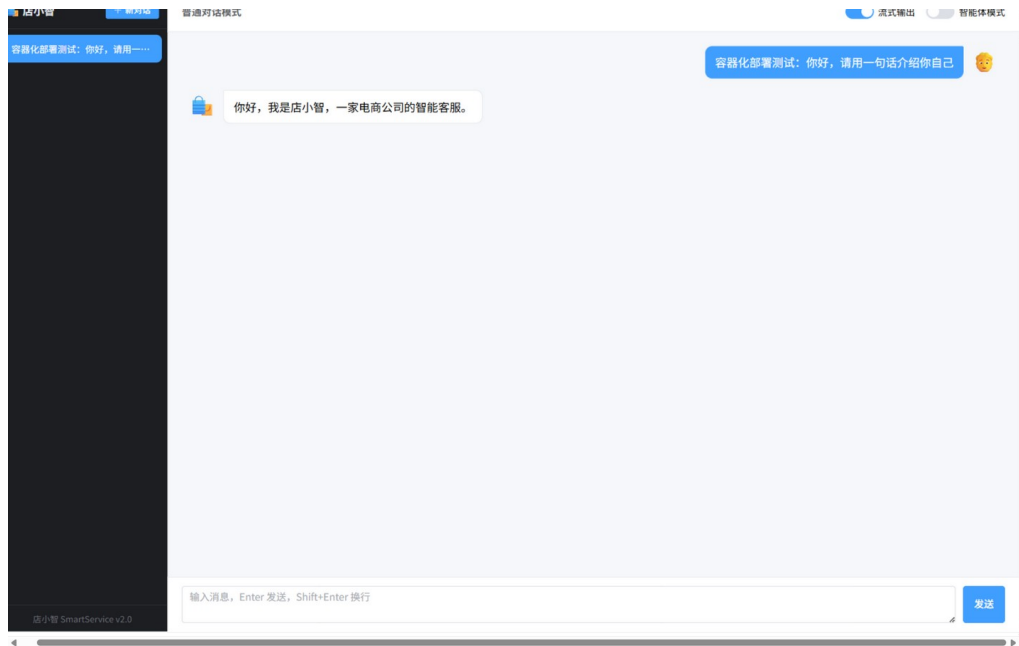


图 10 毁灭测试：docker compose down 后重新 up，历史会话与聊天记录完整保留

部署验证过程：docker compose up -d --build 后两个容器均正常启动；访问 8080 页面功能正常；执行 docker compose down 后再次 up -d（毁灭测试），侧边栏历史会话与聊天记录完整保留，证明数据卷持久化真实生效。

6 项目创新点与难点解决方案

6.1 创新点

创新点一：意图路由统一入口——自动分流替代手动开关。

常规做法是让用户用开关选择“普通对话/智能体”两种模式；本系统在后端用结构化输出对消息做意图分类，自动路由到最合适的处理链路：闲聊走 1 次模型调用的普通链路（响应快、零工具开销），业务问题走智能体链路（多工具往返拿真实数据）。配合关键词规则兜底保证可用性，兼顾了成本、响应速度与体验。

创新点二：多模型分工 + 全链路国产大模型适配。

针对 glm-4-flash 与 glm-4.5-flash 各自的能力差异（前者快但记忆召回弱，后者工具调用与召回稳），通过配置中心实现“对话/意图分类/智能体”三链路按需选型，全部使用免费额度模型，零成本运行；并在适配过程中沉淀了对结构化输出模式、思考痕迹、多轮召回等国产模型兼容性问题的通用解决方案。

创新点三：自定义优惠券工具与自动化评估体系。

在课程要求的 4 个工具之外自行设计了第 5 个业务工具 `get_coupon`（优惠券状态查询，含过期/已用/可用多状态）；并编写自动化评估脚本 `test_eval.py`，以 12 个用例持续度量工具选择正确率与平均延迟，形成“评估→修改→重测”的工程化迭代闭环（实测正确率 100%）。

6.2 难点与解决方案

难点一：SqliteSaver 全量消息流导致的工具记录与记忆问题。

难点描述：接入 SqliteSaver 后，`agent.invoke` 返回的是整个线程的全量消息；一方面统计本轮 `tools_used` 时把历史轮次的工具调用也计入（记录越来越长），另一方面“查完订单再追问订单号”时 `glm-4-flash` 无法从长上下文中正确召回。

原因分析：LangGraph 检查点按 `thread_id` 保存线程完整状态，`invoke` 输入只追加新消息、输出返回全部消息，这是框架设计而非缺陷；召回失败则是小模型能力问题。

解决过程：工具统计改为“只取最后一条 HumanMessage 之后的 AI 消息”并对重复调用去重；用最小复现脚本对比两个模型的召回能力后，把智能体模型切换为 glm-4.5-flash。

最终效果：工具记录与真实调用一致；记忆追问正确答出 DD20240001；12 用例评估 100% 通过。

难点二：国产模型与 LangChain 结构化输出的兼容性。

难点描述：意图路由依赖 with_structured_output 返回强类型对象，但 glm 系列先后出现 JSON 模式输出自然语言、函数调用模式不返回工具调用、Intent 字段带大小写与标点差异等三类不兼容。

原因分析：OpenAI 兼容协议各家实现程度不一，response_format 与 tool_choice 的强制约束在国产平台未被严格执行。

解决过程：逐层排查（裸调 SDK → 不同 method 对比 → 最小复现脚本），最终采用“function_calling 通道 + Pydantic field_validator 归一化 + 关键词规则兜底”三层防护。

最终效果：意图路由在 4 组抽查样本上 100% 正确，且任何分类异常都会被兜底规则接管，接口不再 500。

7 总结与展望

7.1 项目总结

本项目独立完成了企业级电商智能客服系统“店小智”的设计、开发、测试与容器化交付：后端实现多轮对话与 SQLite 持久化、5 工具智能体（ReAct + SqliteSaver 记忆）、结构化输出意图路由、SSE 流式输出、滑动窗口限流与统一响应/日志/异常兜底；前端实现会话侧边栏与聊天面板（流式开关、智能体开关、工具调用展示）；交付侧实现 Docker Compose 一键部署与数据卷持久化，并通过毁灭测试验证自动化评估显示工具选择正确率 12/12=100%、平均延迟 3174ms。通过本项目，我完整实践了“大模型 API 调用 → 智能体工具 → 意图路由/流式 → 评估/限流 → 容器化交付”的企业级开发闭环，对分层架构、配置中心、双库分离等工程规范，以及国产大模型兼容性问题有了切身体会。不足之处：天气与订单仍为模拟数据，缺少用户体系与权限控制，智能体多步编排对弱模型仍有偶发不稳定。

7.2 未来展望

①接入真实数据源：用高德/和风天气 API 替换模拟天气，对接真实订单系统接口并为快递轨迹、售后工单扩展更多工具；

②用户体系与安全增强：引入 JWT 登录鉴权与会话隔离，限流存储从进程内存迁移到 Redis 以支持多实例部署；

③检索增强与可观测性：把商品知识库做成第 6 个工具（RAG 检索），并接入 LangSmith/Langfuse 进行调用链追踪与提示词回归评估；智能体链路也支持流式输出，进一步压缩首字时间。

参考文献

- [1] LangChain. LangChain Documentation (v1.x)[EB/OL]. <https://python.langchain.com/docs/>, 2026-09-20.
- [2] LangGraph. LangGraph Documentation: Agents & Persistence[EB/OL]. <https://langchain-ai.github.io/langgraph/>, 2026-09-22.
- [3] FastAPI. FastAPI Documentation[EB/OL]. <https://fastapi.tiangolo.com/>, 2026-09-20.
- [4] 智谱 AI 开放平台. GLM 大模型 API 文档（OpenAI 兼容接口）[EB/OL]. <https://docs.bigmodel.cn/>, 2026-09-24.
- [5] Vue.js. Vue 3 官方文档[EB/OL]. <https://cn.vuejs.org/>, 2026-09-23.
- [6] Docker Inc. Docker Compose Overview[EB/OL]. <https://docs.docker.com/compose/>, 2026-09-26.

附录（可选）

附录 A 答辩演示脚本（3 分钟）：①普通对话验证多轮记忆（“我叫小明” → “我叫什么”）；②刷新页面，历史恢复（持久化）；③智能体模式查订单（🔧 工具前缀）；④复合问题“现在几点，顺便算 $365*24*60$ ”（多步编排）；⑤闲聊句与业务句分别发送 `/api/smart/chat`，展示意图路由；⑥ `docker compose down && up -d` 后数据仍在（交付能力）。

附录 B 项目地址与复现：代码托管于 GitHub（README 含完整复现步骤）；本地开发 `python -m uvicorn app.main:app --port 8000` + `npm run dev`；容器化 `docker compose up -d --build`。

附录 C 评估脚本输出：`test_eval.py` 全部 12 个用例通过（输出原文见代码仓库 `report_assets/test_eval_output.txt`）。